CPSC-354 Report

Angelina House
Chapman University

September 7, 2025

Abstract

This report documents my learning and progress throughout the Fall 2025 semester in CPSC-354: Programming Languages.

Contents

1 Introduction

This report chronicles my work in CPSC-354: Programming Languages during Fall 2025. Each weekly subsection contains restated homework prompts, fully typeset solutions, and a brief reflection or question that ties the technical material to broader themes.

2 Week by Week

2.1 Week 1

Homework

Discuss the MU puzzle from *Gödel, Escher, Bach* (Chapter 1; see course PDF pp. 41–42). Does the string `MU` have a derivation in the MIU formal system starting from `MI`? Why or why not?

The MU (MIU) System Recap

We begin with the single axiom string `MI`. The system has four production rules:

1. If a string ends in `I`, append a `U` at the end.
2. If the string has the form `Mx`, duplicate everything after `M`: `Mx` $\rightarrow$ `Mxx`.
3. Replace any occurrence of `III` by `U`.
4. Delete any occurrence of `UU`.

From `MI`, we can derive many strings by applying the rules:

`MI`(2)`MII`(2)`MIIII`(3)`MUI`.

This illustrates how the rules expand and compress substrings, but it remains unclear how to end with exactly `MU`.

Let $I(s)$ be the number of `I`'s in a string s. Initially, $I(\texttt{MI}) = 1$, i.e., 1 (mod 3). Examine how each rule affects I modulo 3:

- Rule 1 adds a `U`; I is unchanged.
- Rule 2 doubles the count of `I`'s: $I \mapsto 2I$. Thus if $I \equiv 1 \pmod 3$ then $2I \equiv 2 \pmod 3$.
- Rule 3 subtracts three `I`'s (`III` $\to$ `U`); the residue modulo 3 is unchanged.
- Rule 4 deletes `U`'s only; I is unchanged.

Therefore, starting from $I \equiv 1 \pmod 3$, any derivable string keeps $I \equiv 1$ or $I \equiv 2 \pmod 3$, but never $0 \pmod 3$. The target `MU` has $I(\texttt{MU}) = 0$ ($0 \pmod 3$), so it is *not* derivable. Hence, the MU puzzle has no solution.

The puzzle shows how stepping outside a formal system and using an invariant (here, the count of `I`'s modulo 3) can decisively answer a reachability question that seems open-ended from within the rules.

Reading Summary & Question

Summary. Hofstadter introduces formal systems via the MIU puzzle and contrasts *inside-the-system* rule manipulation with *meta-level* reasoning using invariants. This frames later topics on meaning, proof, and limitations of formal systems.

Question. Are there any situations where the MU puzzle could be true, and how much would we have to change to get to that point?

2.2 Week 2

Homework

Using the course notes on abstract reduction systems (ARS), determine for each of the seven given ARSs whether it is *terminating*, *confluent*, and whether it *has unique normal forms (UN)*. Then, fill the grid of the eight possible (Confluent, Terminating, Has UN) combinations with examples or explain why some combinations are impossible.

Quick Definitions

- **Terminating:** there is no infinite chain $x_0 \to x_1 \to x_2 \to \cdots$.
- **Confluent:** if $x \to^* y$ and $x \to^* z$, then y and z are joinable.
- **Has unique normal forms (UN):** every element has a (and in fact the) unique normal form.

Seven ARSs: Diagrams & Properties

Below, each ARS is shown as a tiny diagram (compiled with `tikz-cd`) followed by its properties and a one–line justification.

1) $A = \varnothing$**.**

(empty set, no nodes or arrows)

Props: Terminating ✓, Confluent ✓, UN ✓. *Reason:* All properties hold vacuously.

2) $A = \{a\},\ R = \varnothing$**.**

a

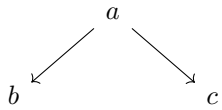
Props: Terminating ✓, Confluent ✓, UN ✓. *Reason:* No arrows; a is a (unique) normal form.

3) $A = \{a\},\ R = \{(a, a)\}$**.**



Props: Terminating ×, Confluent ✓, UN ×. *Reason:* Self–loop gives infinite chain; only one object so forks (if any) are trivially joinable; no normal form exists.

4) $A = \{a, b, c\},\ R = \{(a, b), (a, c)\}$**.**



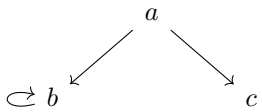
Props: Terminating ✓, Confluent ×, UN ×. *Reason:* From a reach distinct NFs b and c that are not joinable.

5) $A = \{a, b\},\ R = \{(a, a), (a, b)\}$**.**



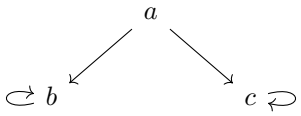
Props: Terminating ×, Confluent ✓, UN ✓. *Reason:* There exists an infinite chain via the loop on a, but all reductions from a can join at b, and every element's normal form is uniquely b.

6) $A = \{a, b, c\},\ R = \{(a, b), (b, b), (a, c)\}$**.**



Props: Terminating ×, Confluent ×, UN ×. *Reason:* Loop at b breaks termination; from a you can reach b (non–NF forever) or c (an NF), so confluence and UN fail.

7) $A = \{a, b, c\},\ R = \{(a, b), (b, b), (a, c), (c, c)\}$**.**



Props: Terminating ×, Confluent ×, UN ×. *Reason:* Loops at b and c; from a diverge to non–joinable loops; no normal forms.

Summary Table (Seven ARSs)

#	Terminating	Confluent	Has UN	Quick reason
1	✓	✓	✓	Empty set: vacuous
2	✓	✓	✓	Single node, no edges
3	×	✓	×	Self–loop; no NF
4	✓	×	×	$a \to b$ or $a \to c$ (two NFs)
5	×	✓	✓	Loop at a but unique NF b
6	×	×	×	Loop at b; $a \to b$ vs $a \to c$
7	×	×	×	Loops at b, c; $a \to b$ vs $a \to c$

Eight Combinations Grid

Confluent	Terminating	Has UN	Example or reason
T	T	T	Example #2 (or #1)
T	T	F	**Impossible:** Confluence+termination ⇒ unique NF
T	F	T	Example #5
T	F	F	Example #3
F	T	T	**Impossible:** UN in a terminating system ⇒ confluent
F	T	F	Example #4
F	F	T	**Impossible:** UN (for all elements) ⇒ confluent
F	F	F	Example #6 or #7

Reflection & Question

Question. What is a real life example of something that has two possible end states like in ARS #4 where a can go to either b or c?

3 Synthesis

4 Evidence of Participation

5 Conclusion

References